PITCHPULSE360

Football Data Engineering and Analytics Platform

A DATA ENGINEERING MINI PROJECT REPORT

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE (AI) & DATA SCIENCE

Submitted by

Name of Student	**Roll Number**
Harsh Birje	225A8127
Aayush Chavanke	225A8128
Aniket Nikam	225A8135
Lokesh Ture	225A8138

Under the guidance of

Prof. Akshay Jadhav

SIES Graduate School of Technology (Autonomous)

RISE WITH EDUCATION

Department of Artificial Intelligence (AI) & Data Science

SIES Graduate School of Technology (Autonomous)

Affiliated to University of Mumbai

Nerul, Navi Mumbai – 400706

A.Y. 2026–27

CERTIFICATE

This is to certify that the Mini Project entitled **"PitchPulse360: Football Data Engineering and Analytics Platform"** is a bona fide work carried out by the following Third Year students:

Student	**Roll Number**
Harsh Birje	225A8127
Aayush Chavanke	225A8128
Aniket Nikam	225A8135
Lokesh Ture	225A8138

The report is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Artificial Intelligence (AI) & Data Science**, University of Mumbai, during the academic year 2026–27.

. . . . . .
Prof. Akshay Jadhav
Faculty Guide

Date: _____ Place: Nerul, Navi Mumbai

ACKNOWLEDGEMENT

We express our gratitude to our guide, **Prof. Akshay Jadhav**, for the opportunity to apply data engineering concepts to a complete mini project. The project connects classroom topics such as data cleaning, relational modeling, SQL analysis, document storage, event processing and workflow automation within one football analytics platform.

We thank the Department of Artificial Intelligence (AI) & Data Science at SIES Graduate School of Technology for the academic environment and resources provided for this work. We also thank our faculty members and classmates for their suggestions and support.

We acknowledge the maintainers of the public football dataset and the open-source tools used in the implementation. Their software and documentation made it possible to develop a local system with inspectable code, repeatable processing and recorded validation results.

Finally, we thank our families and one another for the cooperation required to complete the project and prepare its demonstration.

Project Team
Harsh Birje
Aayush Chavanke
Aniket Nikam
Lokesh Ture

ABSTRACT

Football results are distributed across separate league and season files. A useful analytical application must do more than display these files: it must preserve their origin, correct inconsistent measurements, enforce data rules, store related entities consistently and make calculations repeatable. PitchPulse360 implements this process as a complete local data engineering platform.

The historical dataset contains ***10,781 matches and 30,186 goals*** *across five European leagues and six seasons, from 2018/19 through 2023/24. A Python pipeline profiles and cleans the records, creates stable fixture identifiers and applies a versioned expectation suite before loading PostgreSQL. MongoDB retains source documents and demonstrates flexible document operations. Delta tables organize raw, clean and aggregated outputs into Bronze, Silver and Gold layers. DuckDB supports analytical comparisons and a controlled storage benchmark.*

Reusable SQL views, window functions and multidimensional aggregations support football analysis. Separate generated ticket orders demonstrate daily revenue, monthly growth and repeat-customer metrics. A FastAPI and RabbitMQ path validates and stages match events without changing the historical standings. Prefect coordinates eleven tasks, with retries, logging and an hourly deployment. A Streamlit website presents season filters, standings, team comparisons, records and clearly bounded player demonstrations.

Saved verification reports record 30 passing tests, 45 passing clean-data expectations, nine detected failures in an intentionally damaged sample, and matching analytical results across engines. The implementation covers all fifteen syllabus experiments while distinguishing historical observations, generated examples and local performance measurements.

Keywords: *Data Engineering, Football Analytics, ETL, PostgreSQL, MongoDB, RabbitMQ, DuckDB, Delta Lake, Prefect, Streamlit.*

Contents

CERTIFICATE i

ACKNOWLEDGEMENT ii

ABSTRACT iii

List of Figures vii

List of Tables viii

LIST OF ABBREVIATIONS ix

1 Introduction 1

1.1 Overview 1

1.2 Motivation and Need 1

1.3 Problem Statement 1

1.4 Objectives 2

1.5 Scope and Boundaries 2

1.6 Organization of the Report 2

2 Literature Survey and Technical Background 3

2.1 Relational Modeling and Analytical Queries 3

2.2 Columnar Files and Embedded Analytics 3

2.3 Lakehouse Versioning 3

2.4 Document Storage and Flexible Records 4

2.5 Event Delivery and Workflow Automation 4

2.6 Comparison of Approaches 5

3 Proposed System 6

3.1 Architecture 6

3.2 Components and Responsibilities 7

3.3 Functional Requirements 7
3.4 Non-functional Requirements 7
3.5 Technology and Runtime Selection 8
3.6 Data Separation 8

4 Design and Methodology 9

4.1 Source Collection and Provenance 9
4.2 Exploration and Cleaning 9
4.3 Derived Fields and Fixture Identity 11
4.4 Normalized Relational Model 11
4.5 ETL and Transactional Loading 12
4.6 Lakehouse Layers 13
4.7 Quality Expectations 13

5 Implementation and Website 14

5.1 Analytical SQL and Reusable Metrics 14
5.2 Multidimensional Aggregation 14
5.3 MongoDB Operations 14
5.4 Delta Feature Demonstration 15
5.5 Event Capture and Reliability 15
5.6 Prefect Orchestration 16
5.7 Football Website 16
5.7.1 Team Comparison and Historical Records 18
5.7.2 Player and Relegation Demonstrations 19
5.8 Code Organization 19

6 Results, Testing and Discussion 20

6.1 Data Reconciliation 20
6.2 Website Result Example 21
6.3 Storage and Thread Benchmark 21
6.4 Event and Delta Validation 22
6.5 Testing Method and Evidence Scope 23

6.6 Coverage of the Fifteen Experiments 24

7 Conclusion and Future Scope **25**

7.1 Conclusion 25

7.2 Limitations 25

7.3 Future Improvements 25

References **26**

A Operation and Demonstration **27**

A.1 Start and Stop 27

A.2 Useful Local Addresses 27

A.3 Suggested Demonstration Sequence 28

A.4 Individual Experiment Commands 28

B Evidence Index and Reproducibility **29**

B.1 Report Snapshot 29

B.2 Figure Provenance 30

B.3 Administrative Attachments 30

List of Figures

3.1 Architecture of the football data engineering platform 6

4.1 Cleaning, repair, rejection and the pre-load quality gate 10
4.2 Entity relationships in the core PostgreSQL football schema 12

5.1 API-to-broker event capture and duplicate-safe staging 15
5.2 Football overview: competition and season filters with season KPI cards 17
5.3 Calculated league standings and the points distribution chart 17
5.4 Head-to-head page in the running football website 18
5.5 Season records and historical football summaries 19

6.1 Generated exploration output: match coverage and goal distribution 21
6.2 Saved six-mode benchmark on the repeated fixture workload 22

List of Tables

2.1 Comparison of approaches for the selected workload 5

3.1 Implemented components and their responsibilities 7
3.2 Observed data and generated teaching datasets 8

4.1 Key cleaning and validation rules 11

6.1 Recorded data preparation and validation results 20
6.2 Latest saved benchmark: one warm-up and five measured queries per mode . . 21
6.3 Syllabus-to-implementation traceability 24

B.1 Primary local evidence used in the report 29

LIST OF ABBREVIATIONS

Term	Meaning
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation and Durability
API	Application Programming Interface
BI	Business Intelligence
CLI	Command Line Interface
CRUD	Create, Read, Update and Delete
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
EDA	Exploratory Data Analysis
ETL	Extract, Transform and Load
FK / PK	Foreign Key / Primary Key
IQR	Interquartile Range
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
OLAP	Online Analytical Processing
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface

CHAPTER 1

Introduction

1.1 Overview

PitchPulse360 is a football data engineering mini project that converts historical match files into validated storage, reusable analysis and a browser-based website. The selected domain is familiar: clubs play fixtures, score goals, earn points and finish at different positions in a season. Behind these simple outcomes lies a useful engineering problem. Files use short column names, some measurements can be inconsistent, and the same club occurs in many fixtures and seasons.

The project separates these concerns. Ingestion collects the files, cleaning interprets their values, validation decides whether data is safe to publish, storage represents the accepted records, and the website answers football questions. The same cleaned fixture population is reconciled across the relational database and analytical outputs. Raw documents and source coordinates remain available for inspection.

1.2 Motivation and Need

A chart built directly from one CSV may work for a demonstration, but it becomes difficult to maintain when thirty files, multiple competitions and several analytical questions are involved. A missing value can be mistaken for zero, a club alias can split a team's history, and repeated calculations can produce inconsistent results. A controlled pipeline addresses these problems before the data reaches a chart.

Football also provides clear cross-field checks. Home and away teams should differ. Goals must be non-negative integers. A home win must agree with the score. Shots on target cannot exceed total shots. These relationships make validation understandable and testable rather than a list of arbitrary technical checks.

1.3 Problem Statement

To design and implement a reproducible football data platform that ingests multi-season CSV files, profiles and cleans the records, validates their quality, stores structured and document

data, calculates consistent analytical results, demonstrates event-driven capture and presents historical football insights through a modern website.

1.4 Objectives

1. Preserve all thirty league-season inputs and retain record provenance.
2. Standardize names, dates and numeric values without inventing measurements.
3. Design a normalized PostgreSQL model and demonstrate analytical SQL.
4. Implement repeatable ETL, reusable metrics, OLAP and document CRUD.
5. Demonstrate Delta version history, schema checks and local storage comparisons.
6. Capture validated events through an API and a durable message broker.
7. Automate the workflow with Prefect and retain execution evidence.
8. Present accessible football analysis and document all fifteen experiments.

1.5 Scope and Boundaries

The historical population covers the Premier League, La Liga, Serie A, Bundesliga and Ligue 1 over six seasons. The website is an archive, not a live-score service. Cached files are reused; missing files can be downloaded from Football-Data.co.uk. An hourly run repeats the configured historical workflow and does not extend its season range automatically.

The platform runs locally through Docker Compose. Ticket transactions are generated classroom data, player movement points are generated visual demonstrations and the benchmark repeats fixtures to create a controlled larger workload. Streaming uses real API, broker and database components, while its example events are replayed or manually submitted. Official deductions, league-specific tie-breaks and all qualification exceptions are outside the implemented standings rules.

1.6 Organization of the Report

Chapter 2 reviews the technical approaches and their trade-offs. Chapter 3 presents the proposed architecture and requirements. Chapter 4 explains data preparation, modeling and processing. Chapter 5 documents the website and implementation modules. Chapter 6 presents measured results and syllabus coverage. Chapter 7 concludes with limitations and practical future work. Appendices provide operation commands and an evidence index.

CHAPTER 2

Literature Survey and Technical Background

2.1 Relational Modeling and Analytical Queries

Relational databases represent entities in tables and connect them through keys. PostgreSQL provides joins, constraints, transactions, views and window functions within this model [1]. For this project, clubs, leagues and seasons are reusable entities; a match references them rather than repeating every description as its identity. This supports referential consistency and makes the join paths explicit.

Normalization and analytical convenience serve different purposes. The core football schema is normalized, while a separate partitioned analytical fact stores selected descriptive fields for query experiments. Treating these as separate models avoids claiming that every optimized analytical table is in 3NF. Window calculations retain individual rows while adding rankings, previous-season values or running totals.

2.2 Columnar Files and Embedded Analytics

CSV is easy to exchange but represents values as text and must be parsed when scanned. Parquet stores columns with metadata and compression. DuckDB can query Parquet directly and push selected columns and filters into its reader [2]. The project evaluates these choices with identical prepared rows rather than assuming that every columnar scan is automatically faster.

DuckDB operates on the local machine. Configuring four threads sets an execution limit; it does not create four computers or guarantee that each operation uses all workers. A Spark or Hadoop cluster would add operational complexity that is unnecessary for the selected educational workload. The local benchmark therefore fulfills the syllabus option for parallel batch processing without claiming distributed-cluster performance.

2.3 Lakehouse Versioning

Delta Lake adds a transaction log to data files and supports versioned table state, schema enforcement and historical reads [3]. The project uses the Python delta-rs implementation

through the `deltalake` package. Bronze, Silver and Gold describe processing stages; Delta versions describe changes to one table. These are different concepts.

A separate demonstration table is used to prove versioned insert, update and delete behavior. This isolates the teaching experiment from the historical football archive. Reading version zero through the actual Delta reader provides stronger evidence than merely listing filenames in a log directory.

2.4 Document Storage and Flexible Records

MongoDB represents records as documents. Nested notes and optional scouting fields fit naturally into this structure. The project retains source match documents and separately inserts, reads, updates and deletes a temporary scouting report. Filters, projections and compound indexes show how document access differs from relational joins. The document copy is a source-preservation path, not a replacement for the clean relational model.

2.5 Event Delivery and Workflow Automation

RabbitMQ distinguishes publisher confirms from consumer acknowledgements. The former confirms broker acceptance; the latter confirms processing by a consumer [4]. Neither alone proves that a database contains the event. The application must choose acknowledgement timing and handle redelivery. Here, event identity and commit-before-acknowledgement make repeated delivery safe for staging inserts.

Workflow orchestration manages a different problem: ordering batch tasks, retrying failures and retaining run state. Prefect coordinates the eleven-step pipeline. The broker transports individual events; the orchestrator coordinates the overall batch. They are complementary components rather than interchangeable schedulers.

2.6 Comparison of Approaches

Table 2.1: Comparison of approaches for the selected workload

Approach	Strength	Trade-off
CSV and spreadsheet	Easy to inspect and exchange	Manual rules are difficult to reuse consistently
Pandas-only notebook	Flexible exploration	Needs additional structure for services and repeatable operation
PostgreSQL	Relationships, constraints and transactional loads	Requires a running server
MongoDB	Flexible nested source documents	Does not replace the relational model's join rules
DuckDB and Parquet	Compact local analytics and direct file queries	Not a distributed cluster
Delta tables	Versioned files and historical reads	Requires transaction-log-aware readers
API with RabbitMQ	Buffered, decoupled event capture	Adds delivery and failure-handling responsibilities

CHAPTER 3

Proposed System

3.1 Architecture

The platform contains five responsibilities: collection, controlled transformation, storage, analysis and presentation. Historical CSV files enter the batch pipeline. Python preserves a raw snapshot and produces cleaned records. A quality gate runs before PostgreSQL loading. MongoDB stores source documents, while Delta tables publish the raw, cleaned and aggregated lakehouse layers. SQL and DuckDB experiments generate inspectable reports. The website reads the prepared analytical data.

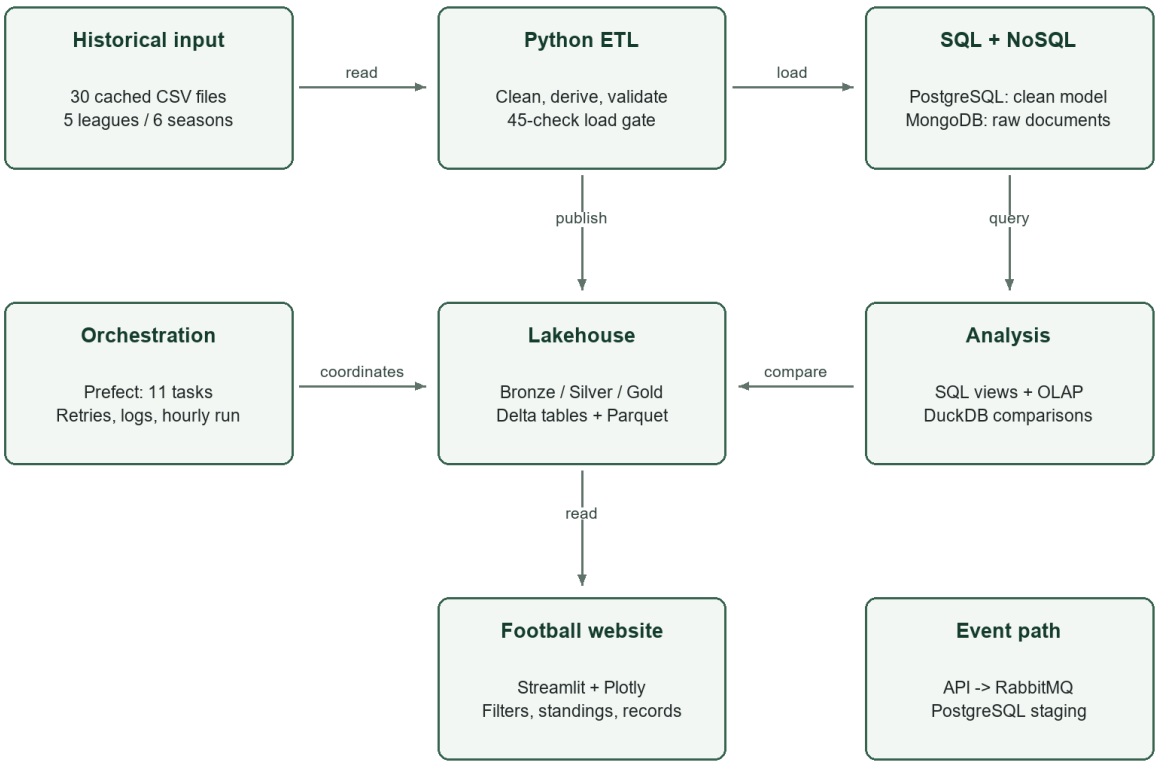


Figure 3.1: Architecture of the football data engineering platform

The event route is deliberately separate. FastAPI publishes an accepted payload to RabbitMQ; a consumer validates and stages it in PostgreSQL. Staged events can be inspected through the

API. They do not immediately change the completed historical results or the website's league table.

3.2 Components and Responsibilities

Table 3.1: Implemented components and their responsibilities

Component	Role
Python / Pandas	Collection, cleaning, profiling, derived fields and validation
PostgreSQL	Normalized fixture model, constraints, views and event staging
MongoDB	Source documents, nested scouting demonstration and indexed reads
Delta / Parquet	Bronze, Silver and Gold tables; versioned demonstration and file outputs
DuckDB	OLAP comparison and controlled storage/thread benchmark
FastAPI	Typed event capture, status and report endpoints
RabbitMQ	Durable event transport between publisher and consumer
Prefect	Eleven task states, retries, logs and hourly deployment
Streamlit / Plotly	Football filters, tables, comparisons and interactive visualizations
Docker Compose	Repeatable startup of databases and application services

3.3 Functional Requirements

The system must read every configured league-season file, reject incomplete source coverage, preserve original record coordinates, clean deterministic defects, stop publication when the quality gate fails, load accepted data transactionally, generate consistent analytical results and expose a usable football interface. It must also demonstrate document CRUD, schema rejection, time travel, queued event capture and a scheduled workflow.

3.4 Non-functional Requirements

Reproducibility is supported by locked direct package versions, stable fixture keys and saved evidence. Reliability is supported by task retries, a workflow lock, database constraints and duplicate-safe event inserts. Maintainability is supported by separating Python modules, SQL

models, validation rules and the website. The implementation is designed for local teaching and review; its demonstration credentials and single-machine storage do not constitute a production deployment design.

3.5 Technology and Runtime Selection

The pinned application dependencies include Pandas 2.3.3, DuckDB 1.4.1, deltalake 0.25.5, Streamlit 1.50.0, FastAPI 0.118.0 and Prefect 3.4.25. These are project versions, not claims about the newest releases. Docker Desktop supplies the local container engine on Windows. The launch scripts start services, wait for readiness and open the dashboard at port 8502. Initial image and dependency downloads require internet access.

3.6 Data Separation

Table 3.2: Observed data and generated teaching datasets

Population	Size	Meaning
Historical fixtures	10,781 matches	Source results across five leagues and six seasons
Ticket orders	6,000 orders	Generated sales examples linked to historical match IDs
Benchmark workload	250,000 rows	Repeated fixtures for a controlled processing comparison
Player profiles/actions	34 / 5,741	Curated profiles with generated movement examples
Event demonstration	20 valid plus two probes	Replayed valid data, one duplicate and one invalid message

Keeping these populations separate prevents a visual demonstration from being mistaken for measured real-world behavior. In particular, ticket revenue is not real club revenue and generated heatmaps are not tracking-provider observations.

CHAPTER 4

Design and Methodology

4.1 Source Collection and Provenance

The ingestion module configures five competition codes: E0, SP1, I1, D1 and F1. Each is read across six season codes, producing thirty source files. Existing cached CSV files are reused; a missing file is requested from Football-Data.co.uk. If any configured edition is empty, the active extraction task raises an error instead of publishing a partial archive.

The original column names are mapped to readable names. For example, FTHG becomes `home_goals`, FTAG becomes `away_goals`, and FTR becomes `result`. League, country, season, source filename and source row are added. The raw snapshot is saved before cleaning, allowing a processed row to be traced back to its origin.

4.2 Exploration and Cleaning

Profiling records row counts, types, null counts, duplicates and numeric summaries before and after cleaning. Text fields are trimmed and repeated whitespace is collapsed. Known club aliases are normalized. Dates are parsed with day-first handling and invalid conversions become missing values for validation. Numeric values are converted with explicit integer and range rules.

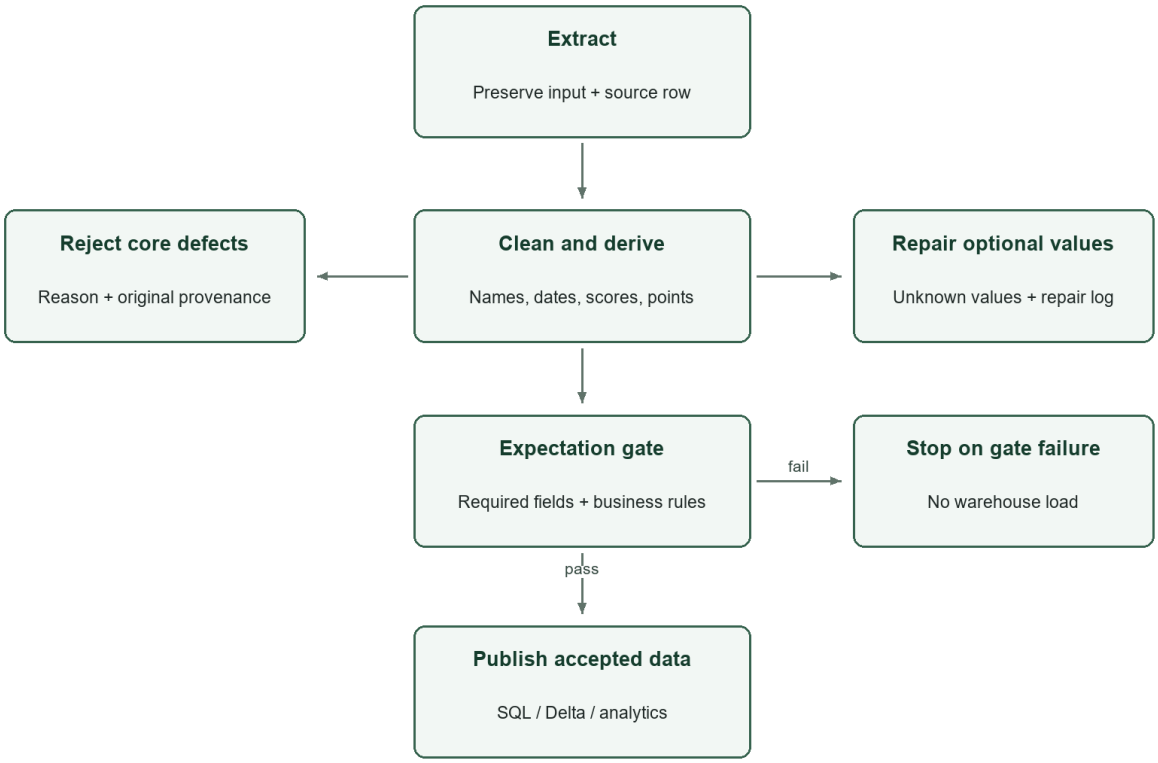


Figure 4.1: Cleaning, repair, rejection and the pre-load quality gate

The policy distinguishes essential facts from optional measurements. An invalid goal score makes the fixture unsuitable for publication. An inconsistent optional shot measurement can be cleared to unknown while preserving a valid score. The repair is recorded in a separate audit file. A missing shot count is not filled with zero because zero would assert that no shots occurred.

Table 4.1: Key cleaning and validation rules

Issue	Rule	Treatment
Blank identifiers	Teams, league, country and season required	Reject core-defect row with reason
Invalid dates	Must parse to a valid date	Reject row
Goal values	Integer in allowed non-negative range	Reject impossible or missing score
Optional statistics	Valid integer/range and cross-field consistency	Clear invalid measurement; log repair
Result code	Must agree with final goals	Derive from score and count repair
Duplicate fixture	Unique league, season, date and team pair	Keep first; quarantine duplicate
Unusual valid score	Total goals above IQR threshold	Flag for inspection; retain

4.3 Derived Fields and Fixture Identity

Each accepted fixture receives a deterministic identifier derived from league, season, date, home club and away club. The implementation hashes this combination with SHA-256 and keeps the first 24 hexadecimal characters. Reprocessing the same fixture produces the same identity. A uniqueness assertion detects a conflicting result in the processed population.

The score determines H, D or A for home win, draw or away win. Total goals are home goals plus away goals. Each winner receives three points and each team receives one point for a draw. Goal difference is goals scored minus goals conceded. These fields supply both analytical SQL and website calculations.

4.4 Normalized Relational Model

The core model contains countries, leagues, seasons, league editions, teams and matches. The fixture grain is one row per match. A league edition identifies a unique league-season combination. The matches table references that edition and references teams twice, once for home and once for away.

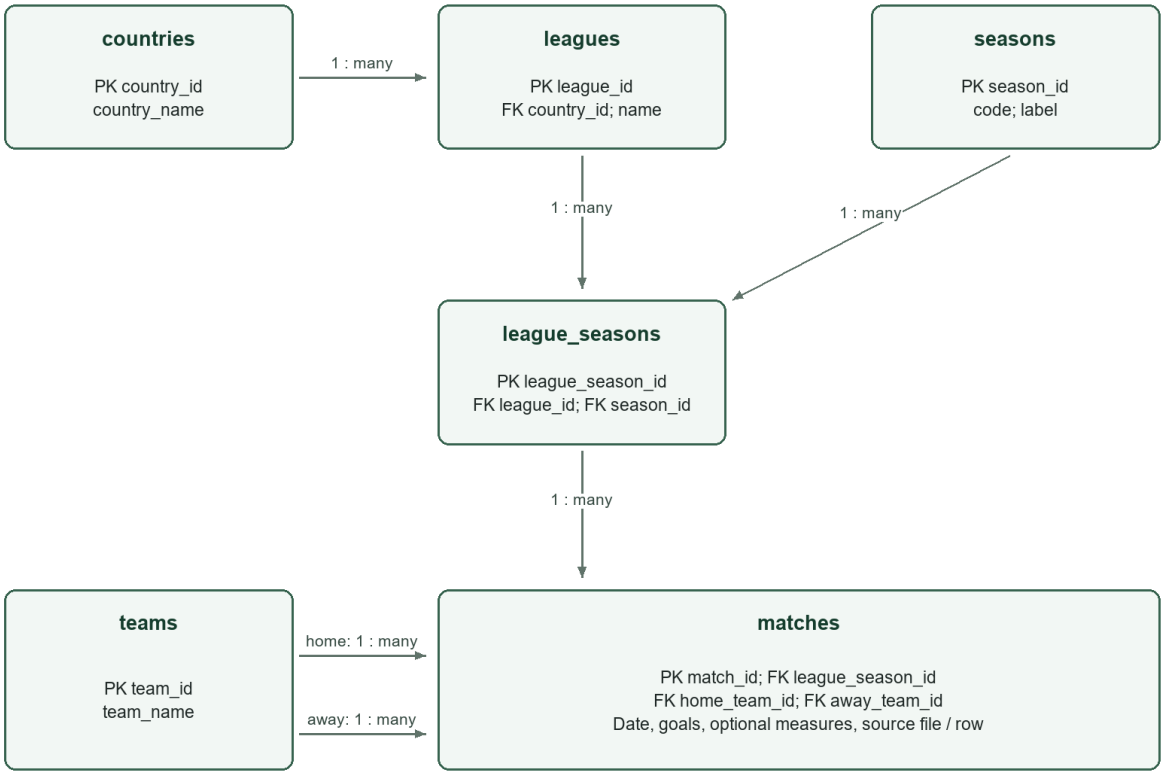


Figure 4.2: Entity relationships in the core PostgreSQL football schema

First normal form keeps individual fields atomic. Separating recurring club, league and season descriptions avoids repeated descriptive entities. Third normal form is supported by moving the league's country relationship out of each match: the country is reached through the league. A team's name depends on its team key, while match measures depend on the match identity.

Primary keys, foreign keys, a unique fixture combination and score/shot constraints enforce this model. A separate `analytics.match_facts` table is partitioned by season code for analytical experiments. It intentionally repeats selected descriptions and is not presented as part of the normalized core.

4.5 ETL and Transactional Loading

The clean Parquet output is the accepted input to the database load. The loader resolves descriptive keys, uses PostgreSQL COPY for match data and refreshes its snapshot transactionally. A failure within the transaction prevents a partly refreshed relational snapshot from being committed. ETL audit records store the run identity and accepted/rejected counts.

The overall platform writes to several independent systems. A PostgreSQL transaction does not create a cross-system transaction across MongoDB, Delta and report files. A failed later task can leave earlier task outputs in place. Prefect exposes that failed flow state; the workflow lock prevents concurrent manual and scheduled runs from replacing outputs together. The reports are therefore associated with run identifiers, not assumed to be one globally atomic commit.

4.6 Lakehouse Layers

Bronze stores the raw snapshot with ingestion metadata. Silver stores validated match rows, calculated points and goal differences. Gold aggregates club-season standings. Each layer is stored as a Delta table backed by Parquet and partitioned by league. The dashboard reads these prepared layers; it does not run a full relational ETL for every chart interaction.

4.7 Quality Expectations

The versioned JSON suite describes required columns, non-null and non-blank fields, data types, unique keys, value ranges, allowed codes and business consistency. The implementation evaluates these rules with Pandas and writes a per-check JSON result and an HTML report. It is a custom expectation suite, not a claim that the Great Expectations package is used.

The clean dataset must pass before loading. A separate intentionally damaged sample is evaluated to prove that the suite detects defects. Those sample failures are expected test outcomes; they are not loaded into the historical warehouse. The proportion of passing expectations measures checks, not the percentage of valid records.

CHAPTER 5

Implementation and Website

5.1 Analytical SQL and Reusable Metrics

The enriched-match view joins fixtures to readable team, league and season names. A team-match view expands each fixture into a home-team contribution and an away-team contribution. Grouping those contributions produces club-season played matches, points, goals for and goals against. DENSE_RANK supplies a ranking, LAG and LEAD support season comparisons, and a windowed SUM produces cumulative points.

Reusable views define daily goals and team win percentage on historical football results. To satisfy the revenue-oriented syllabus examples, a separate generated ticket-order table defines daily revenue, monthly sales growth and repeat-customer percentage. Refunded orders do not count as paid revenue or purchases. The month calendar retains zero-sales months. Growth is undefined when there is no previous month or its revenue is zero.

The implementation compares each business view with equivalent direct SQL and saves query plans and repeated timings. An ordinary view saves a definition; it does not store a precomputed answer or guarantee that the query becomes faster. The reported 46.4 percent repeat-customer measure belongs only to the generated ticket population.

5.2 Multidimensional Aggregation

ROLLUP over league and season produces edition detail, league subtotals and an overall total. CUBE also includes season-only totals. GROUPING SETS explicitly requests selected combinations. With five leagues and six seasons, the saved ROLLUP contains 36 rows and the two-dimension CUBE contains 42. PostgreSQL and DuckDB execute equivalent aggregations, and their answers are compared before timings are accepted.

5.3 MongoDB Operations

Source documents are upserted with a key built from source filename and row number. Compound indexes support league-season and home-team/date access. A temporary nested scouting document demonstrates insertion, projection-based reading, a rating and nested-note update,

and deletion followed by an absence check. This document is separate from historical match results. The saved explain output records the indexed source query.

5.4 Delta Feature Demonstration

The isolated table begins with two rows at version zero. Appending a third row creates version one. Updating the first row's goals creates version two, and deleting the second row creates version three. Reading version zero returns the original two records through the Delta reader. Two deliberately incompatible writes, one with wrong columns and one with a nonnumeric goal value, are rejected; the current version remains unchanged.

5.5 Event Capture and Reliability

The REST route validates a match event before publishing it. A confirmed broker publication returns HTTP 202 Accepted. This means the broker accepted the event, not that the consumer has already committed it. Invalid request fields or score/result disagreement fail validation. If the broker cannot confirm publication, the API returns an unavailable response.

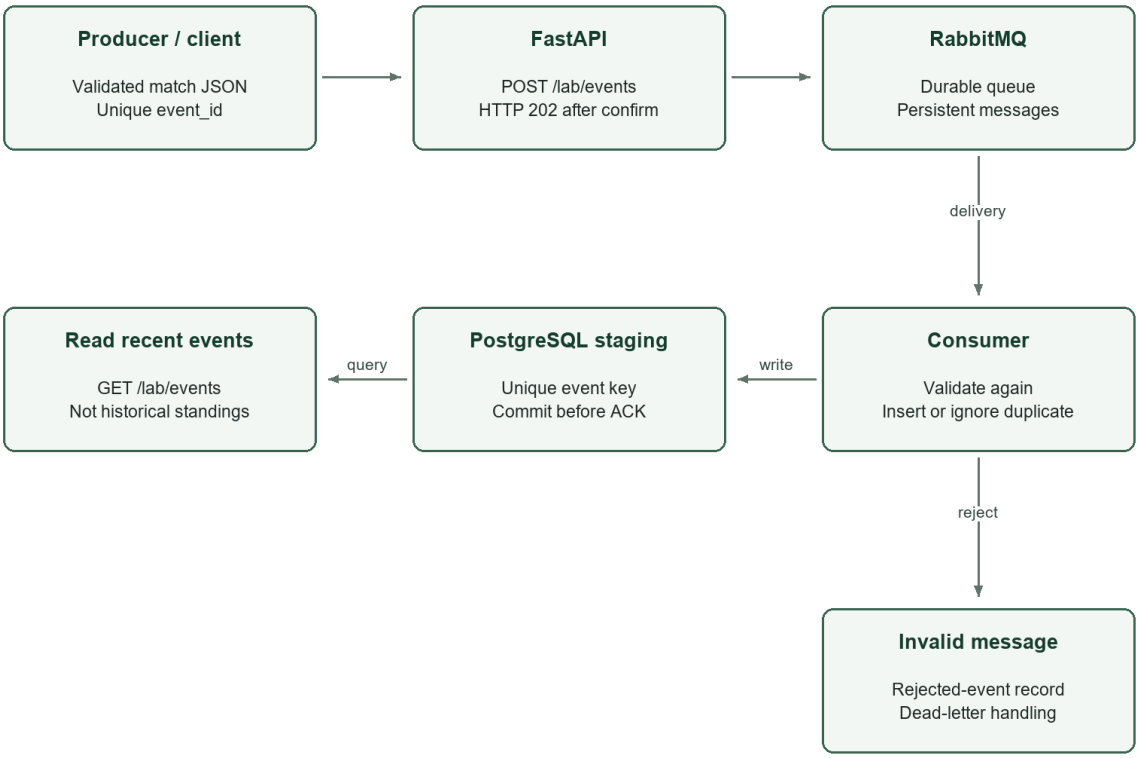


Figure 5.1: API-to-broker event capture and duplicate-safe staging

The consumer validates again, commits a unique event to staging and acknowledges successful processing. Repeated event IDs do not create a second staging row. Invalid messages are recorded and handled through the rejection/dead-letter path; transient processing errors can be retried. This is at-least-once delivery with an idempotent database effect, not a general exactly-once claim.

5.6 Prefect Orchestration

The active flow executes extraction, cleaning, expectations, PostgreSQL, metric views, MongoDB, OLAP, lakehouse publication, Delta demonstration, benchmarking and streaming verification. These are eleven actual tasks. The current implementation invokes them in sequence; it should not be described as running all branches concurrently.

Retries differ by task: validation failures are not blindly retried, while transient extraction or database failures can receive configured retries. A file lock serializes overlapping runs. The deployment uses a 3,600-second interval and a run limit of one. Docker and the scheduler must remain running for future scheduled execution. The archive does not become a live football feed merely because the pipeline is scheduled.

5.7 Football Website

The website opens with a competition and season selector. Its header identifies the selected edition and its KPI cards show matches, goals, home win rate and calculated table leader. The visual design uses green, gold and neutral surfaces, with native light/dark appearance. The layout adapts its cards and tables for smaller screens.

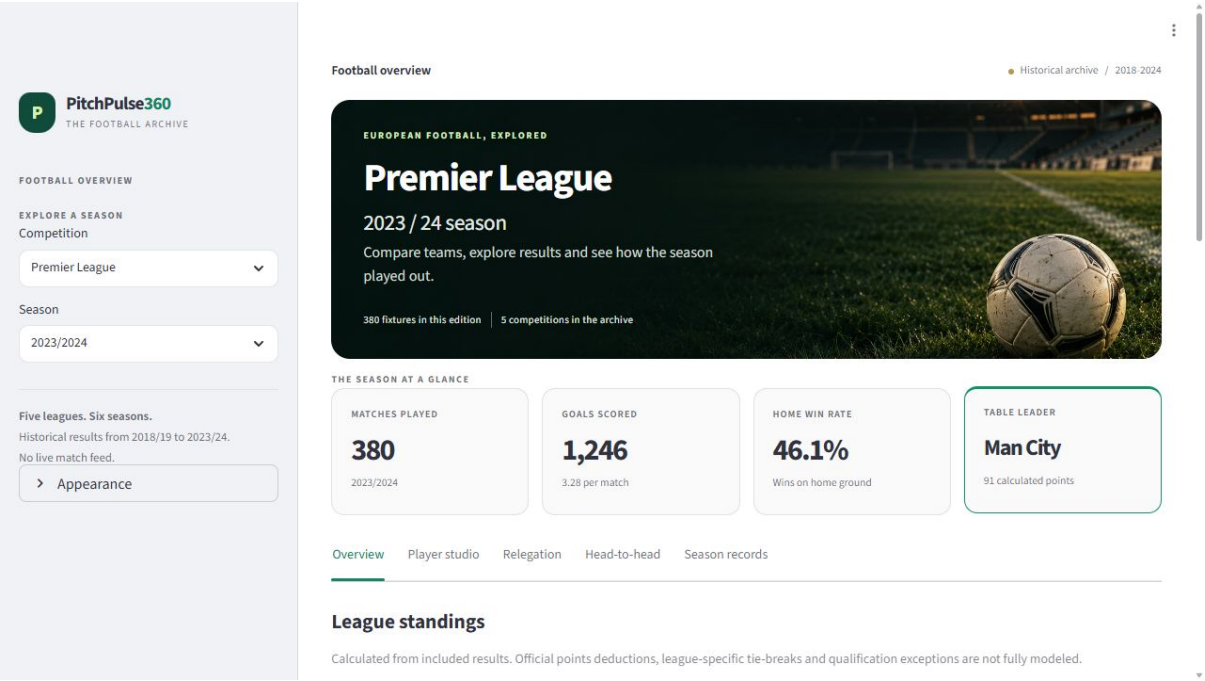


Figure 5.2: Football overview: competition and season filters with season KPI cards

The Overview tab contains standings, points distribution and an attacking-versus-defensive chart. Standings expose played, won, drawn, lost, goal difference and points by default, with additional fields available through the column menu. A club's points are calculated as three times wins plus draws. The table is a result-derived comparison, not an official league adjudication.

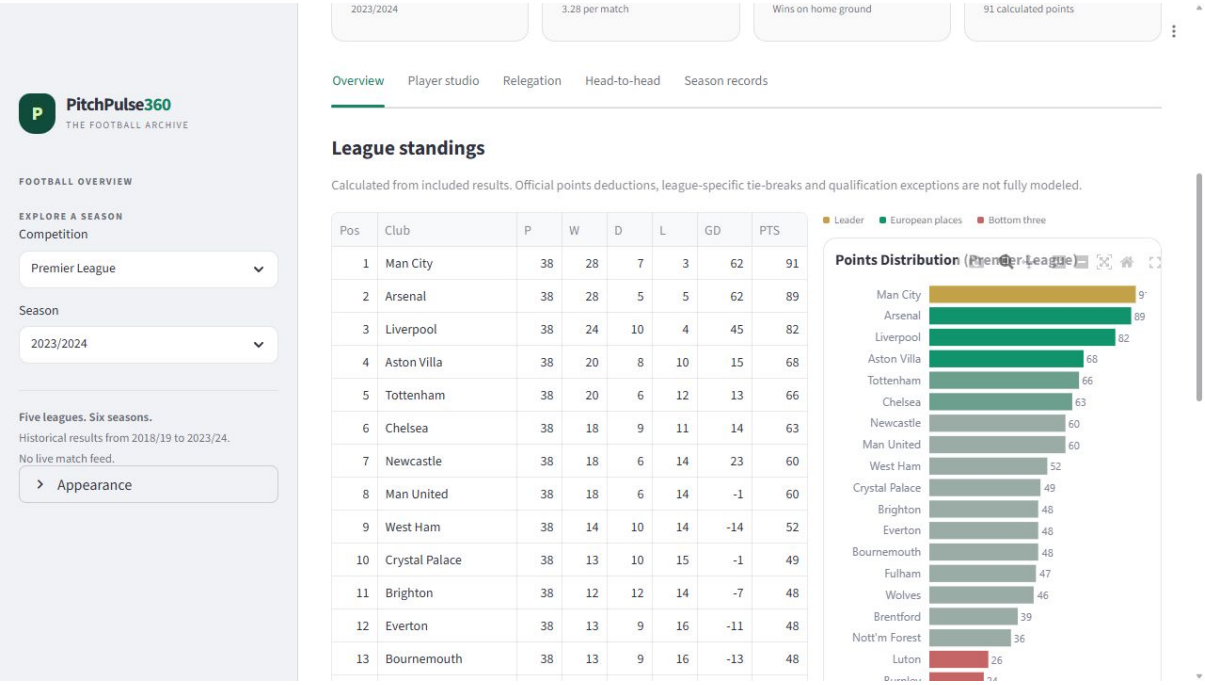


Figure 5.3: Calculated league standings and the points distribution chart

5.7.1 Team Comparison and Historical Records

Head-to-head analysis compares two clubs using the selected historical population. It presents their meetings and performance summaries, while season trajectories place the results in a wider context. The records page identifies high-scoring fixtures and summarizes home/away outcomes across the archive. These controls read already prepared data and recalculate the selected views.

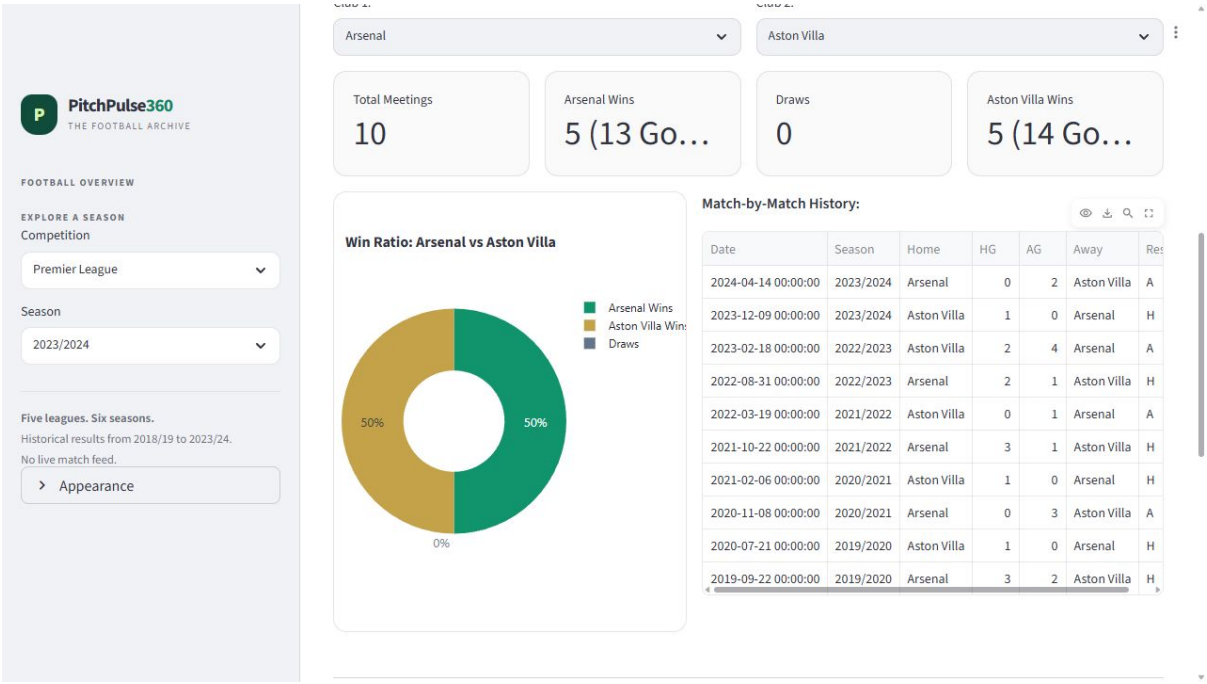


Figure 5.4: Head-to-head page in the running football website

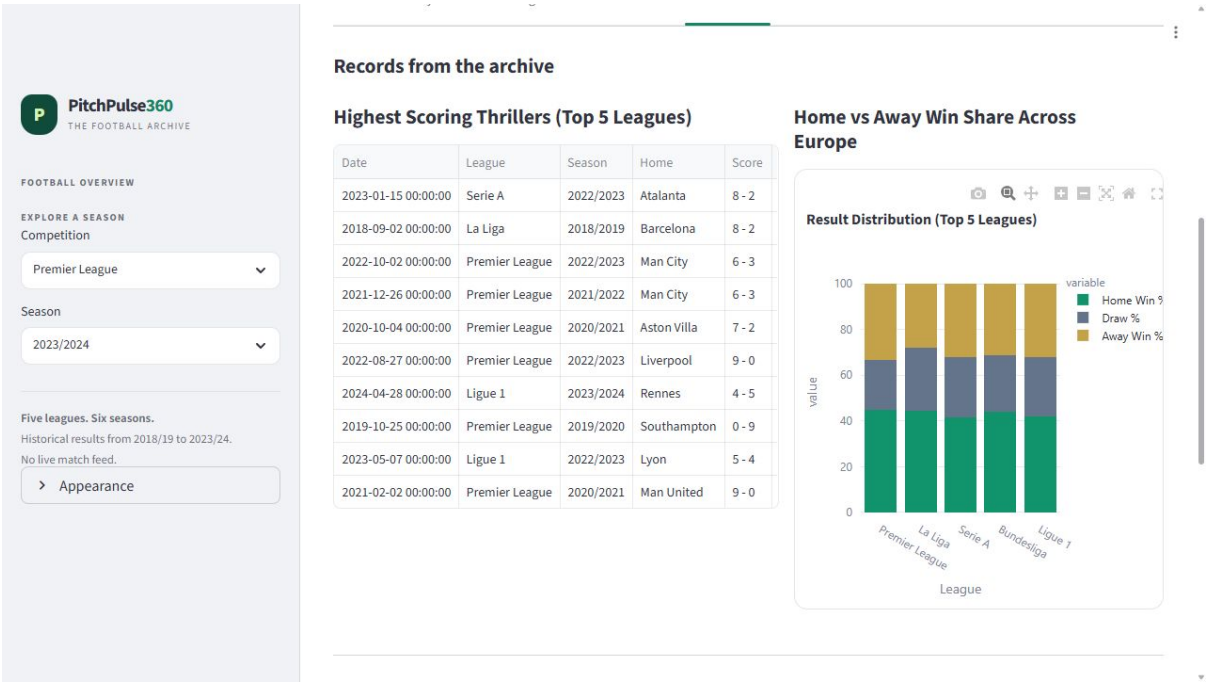


Figure 5.5: Season records and historical football summaries

5.7.2 Player and Relegation Demonstrations

The Player studio combines profiles with visual movement demonstrations. Generated action locations support heatmaps and positioning examples; they must not be interpreted as real tracked player events. The relegation page highlights the bottom end of calculated standings. Its labels follow simplified position-based rules, so it cannot settle official deductions, play-offs or exceptions for every league.

5.8 Code Organization

The `platform_lab` package contains cleaning, relational loading, metrics, document operations, aggregation, expectations, benchmarking, streaming, verification and orchestration. The `models` directory contains SQL definitions; `quality` contains the expectation suite; `storage` contains lakehouse publication; `dashboard` contains the website. The `results` directory holds machine-readable outputs, charts, plans and HTML reports. This separation makes a displayed number traceable to a calculation and a saved result.

CHAPTER 6

Results, Testing and Discussion

6.1 Data Reconciliation

The saved run reports 10,781 historical matches, 30,186 goals, 140 distinct teams, thirty league editions and 586 club-season summaries. PostgreSQL and MongoDB reconcile to 10,781 match rows/documents. Not every league edition contains the same number of matches; this includes the interrupted 2019/20 Ligue 1 season. The dataset is treated as the supplied historical coverage rather than padded to an assumed uniform total.

Table 6.1: Recorded data preparation and validation results

Measure	Recorded value
Input / accepted fixture rows	10,781 / 10,781
Rejected core-defect rows	0
Rows with optional measurement repair	1
High-scoring outliers retained	69
Total historical goals	30,186
Clean expectations passed	45 / 45
Failures detected in separate damaged sample	9
Saved automated tests passed	30
Completed tasks in verified flow	11

One fixture has inconsistent away-shot measurements; the relevant optional values are cleared and audited while its valid score remains. No core fixture is rejected in this input snapshot. This is different from claiming that every raw field was perfect. The outlier rule flags total goals above seven, producing 69 valid unusual fixtures retained for inspection.

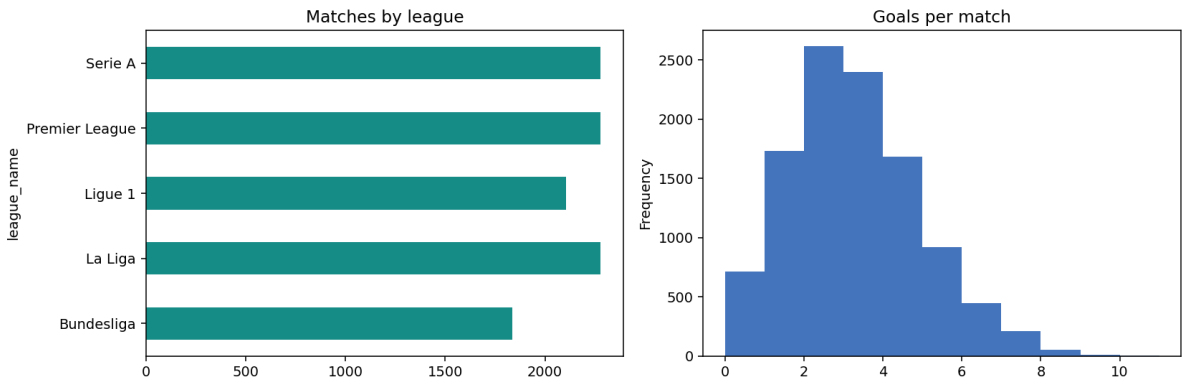


Figure 6.1: Generated exploration output: match coverage and goal distribution

6.2 Website Result Example

For the selected Premier League 2023/24 edition, the website shows 380 matches, 1,246 goals and a 46.1 percent home win rate. The calculated table leader is Manchester City with 91 points. These values can be recomputed from the selected results. A table position based on those results may differ from official standings where points deductions or additional rules apply.

6.3 Storage and Thread Benchmark

The benchmark repeats accepted fixtures to form exactly 250,000 rows. Identical prepared data is written to CSV, partitioned ZSTD Parquet and a native DuckDB table. Each storage mode receives a warm-up and five timed runs under one-thread and four-thread limits. Result equality is checked for every mode.

Table 6.2: Latest saved benchmark: one warm-up and five measured queries per mode

Storage	Thread limit	Median (ms)	Equal answers
CSV	1	187.593	Yes
Parquet	1	324.684	Yes
DuckDB	1	11.086	Yes
CSV	4	223.638	Yes
Parquet	4	132.689	Yes
DuckDB	4	11.078	Yes

The latest saved run shows that four-thread CSV is slower than one-thread CSV, while four-thread Parquet improves on its one-thread measurement. This illustrates why performance must be measured. Startup overhead, file access, the container bind mount and the selected workload can affect the result. The values are local warm-cache observations, not universal engine rankings.

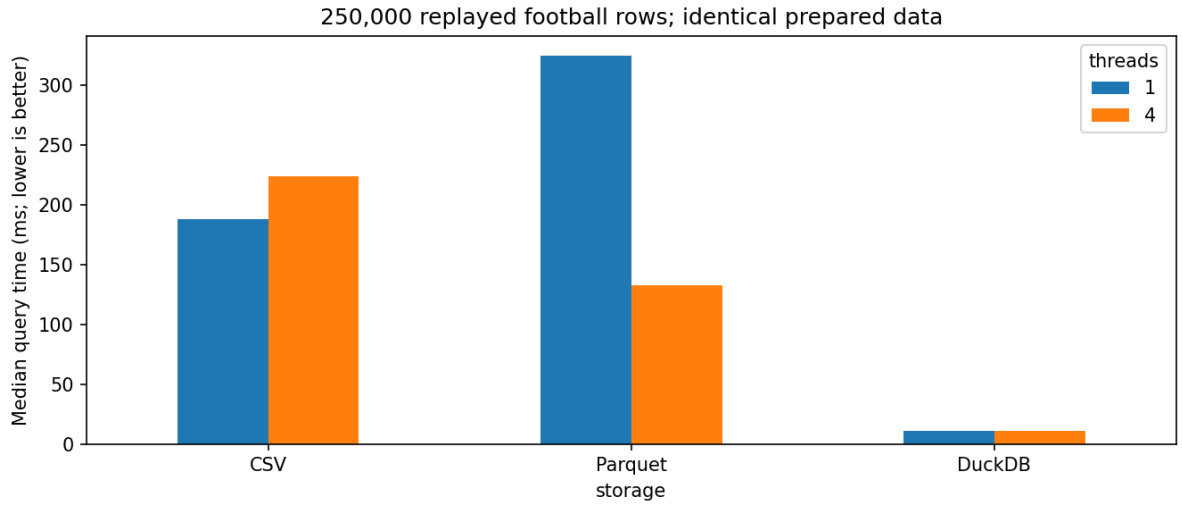


Figure 6.2: Saved six-mode benchmark on the repeated fixture workload

The filtered Parquet plan reads one of five league files. Partition pruning demonstrates skipped irrelevant partitions rather than a claim that every query reads only one file. Repeated fixture values also compress unusually well, so their size ratio should not be predicted for newly observed independent data.

6.4 Event and Delta Validation

The saved event demonstration publishes twenty valid messages, one duplicate and one invalid message. It records twenty unique staged rows and one rejection for that run. The duplicate is ignored. A dead-letter queue depth is cumulative broker state and should not be interpreted as the number of invalid records in one demonstration.

Delta verification records versions zero through three, a successful read of version zero and two rejected incompatible writes. The separate table's version does not advance after those failures. Metric views match their equivalent direct SQL, and all three OLAP forms match across PostgreSQL and DuckDB. These checks test actual behavior rather than the existence of module names.

6.5 Testing Method and Evidence Scope

The completion summary records thirty passing automated tests, including service integration and deliberate defect cases. Reconciliation compares stored counts and goal totals. Quality tests evaluate both valid data and a separate damaged sample. API tests cover accepted and invalid requests; event tests cover duplicate-safe storage and rejection. UI checks exercise all five football tabs and league/season changes.

The report was prepared on 16 October 2026 using saved execution artifacts and screenshots from the running product. It does not claim that every saved historical test was rerun during document preparation. The saved verification run and the latest batch can have different run identifiers: a later scheduled run can update the latest-result files after verification. Appendix B distinguishes these evidence records.

6.6 Coverage of the Fifteen Experiments

Table 6.3: Syllabus-to-implementation traceability

No.	Experiment	Implemented behavior and evidence
1	EDA and cleaning	Profiles, conversion, null handling, audited repairs, outliers and charts; results/eda.
2	Relational design and SQL	3NF core, keys, constraints, joins, windows, indexes and partition plans; models and results/postgres.
3	Mini ETL	CSV extraction, clean Parquet and transactional PostgreSQL COPY load with audit records.
4	Reusable SQL metrics	Daily revenue, monthly growth and repeat-customer views on labeled generated tickets; direct-SQL equality.
5	Data cubes	ROLLUP, CUBE and GROUPING SETS; matching PostgreSQL/DuckDB results and saved plans.
6	Analytics dashboard	Five football tabs, competition/season filters, KPI cards, tables and charts.
7	MongoDB CRUD	Raw upserts, nested scouting CRUD, filters, projections, indexes and query-plan evidence.
8	Delta Lake features	Create, append, update, delete, old-version read and two incompatible write rejections.
9	REST data capture	FastAPI POST/GET, typed payload validation and broker-backed staging.
10	Event-driven pipeline	RabbitMQ producer/consumer, durable transport, commit-before-ACK and deduplication.
11	Parallel local batch	250,000 repeated rows; CSV/Parquet/DuckDB with one/four-thread settings and equal-result checks.
12	Workflow orchestration	Eleven Prefect tasks, retries, logs, workflow lock and hourly deployment.
13	Data-quality expectations	Versioned custom suite, 45 clean passes, nine expected damaged-sample failures and HTML reports.
14	End-to-end batch	Connected extraction, cleaning, validation, transformation and publication with logs and provenance.
15	Combined mini platform	SQL/NoSQL storage, APIs, validation, ETL, events and website start together through Compose.

CHAPTER 7

Conclusion and Future Scope

7.1 Conclusion

PitchPulse360 connects all fifteen experiments through a complete football data workflow. Historical files become validated records, a normalized relational model, preserved documents, versioned analytical storage and an interactive website. The project demonstrates more than chart creation: it records where data came from, which values were repaired, how metrics were defined and whether execution completed. Stable fixture identifiers, saved plans, quality results and task logs make the final football analysis traceable to its inputs and calculations.

7.2 Limitations

The archive ends at 2023/24 and is not a live feed. Official league rules are simplified. Ticket orders, player movements and benchmark replays are teaching examples. The local services use demonstration credentials and do not share one distributed transaction. Queued events enter staging without automatically changing historical standings. Benchmark results depend on the local machine and storage. Passing the saved tests establishes the demonstrated behavior, while broader operational reliability still requires additional recovery, security and scale testing.

7.3 Future Improvements

1. Add a licensed current-results source with explicit refresh timestamps and source attribution.
2. Implement league-specific deductions, tie-breaks, play-offs and season-dependent qualification rules.
3. Replace generated player movement with licensed event or tracking data when available.
4. Reconcile staged events with stable source match identifiers before publishing refreshed standings.
5. Add incremental processing, immutable run directories and a publication manifest to reduce partial multi-system snapshots.
6. Add access control, managed secrets, backup recovery tests and monitoring before wider deployment.
7. Extend benchmark experiments to larger independent datasets and controlled cold-cache conditions.

Bibliography

[1] PostgreSQL Global Development Group. *PostgreSQL Tutorial: relational concepts, SQL and advanced features*. https://www.postgresql.org/docs/current/tutorial.html. Accessed 16 October 2026.

[2] DuckDB Foundation. *Reading and Writing Parquet Files*. https://duckdb.org/docs/stable/data/parquet/overview. Accessed 16 October 2026.

[3] Delta Lake Project. *Delta Lake Documentation*. https://docs.delta.io/. Accessed 16 October 2026. The project uses the Python delta-rs implementation, not a Spark cluster.

[4] RabbitMQ. *Consumer Acknowledgements and Publisher Confirms*. https://www.rabbitmq.com/docs/confirms. Accessed 16 October 2026.

[5] Football-Data.co.uk. *Historical football results CSV archive*. Dataset URL pattern used by ingestion: https://www.football-data.co.uk/mmz4281/2324/E0.csv. Historical cached files are the input evidence for this project.

[6] PitchPulse360 project team. *Active implementation and executable models*: platform_lab modules, models/postgres_schema.sql, models/metric_views.sql, quality/football_expectations.json, storage/lakehouse_manager.py and dashboard/app.py. Local project snapshot, 16 October 2026.

[7] PitchPulse360 project team. *Saved execution evidence*: results/last_run.json, verification.json, completion_summary.json, eda, metrics, olap, mongo, quality, delta_lab, benchmark, streaming and prefect reports. Local project snapshot, 16 October 2026.

CHAPTER A

Operation and Demonstration

A.1 Start and Stop

Extract the project ZIP completely and keep the folder intact. On Windows, start Docker Desktop and run `Start.cmd`. The full launcher builds or starts services, initializes the pipeline and verifies the platform. `Open_Football.cmd` provides quick startup using prepared data. The installed desktop run file points to the project location; reinstall it if the extracted folder is moved.

```
# From the extracted project directory
.\Start.ps1 -Action Start
.\Start.ps1 -Action Run
.\Start.ps1 -Action Test
.\Start.ps1 -Action Stream
.\Start.ps1 -Action Status
.\Start.ps1 -Action Stop
```

Stop preserves database volumes. Initial setup requires network downloads. Future scheduled runs require the engine, scheduler and dependencies to remain running. Do not use a factory reset as a normal project-start step.

A.2 Useful Local Addresses

Service	Address
Football website	`http://127.0.0.1:8502/`
API documentation	`http://127.0.0.1:8002/docs`
Prefect	`http://127.0.0.1:4201/`
RabbitMQ management	`http://127.0.0.1:15673/`

A.3 Suggested Demonstration Sequence

Open the football website and explain the selected competition, season and KPIs. Change the league and confirm that its fixture count and table change together. Open Head-to-head and Season records. Then demonstrate the API event route and inspect recent staging records through GET `/lab/events`. Finally, show the saved quality HTML, benchmark chart, Delta snapshots and a completed Prefect run. This follows the data from input to result while keeping football insights separate from generated teaching examples.

A.4 Individual Experiment Commands

```
docker compose run --rm pipeline metrics
docker compose run --rm pipeline delta
docker compose run --rm pipeline benchmark
docker compose run --rm pipeline quality
docker compose exec dashboard python -m platform_lab.ui_check
```

These commands regenerate experiment evidence and can change current timings. Use the saved run identifiers when referring to the measurements in this report.

CHAPTER B

Evidence Index and Reproducibility

B.1 Report Snapshot

The latest saved batch identifier is `01a1412f-4299-7350-98ef-3bb67828d493`. Its recorded duration is 37.991 seconds. The benchmark table in Chapter 6 is read from that same saved latest-run artifact; older narrative reports may contain measurements from earlier runs.

The verification artifact separately records `01a1412d-46ac-7abf-a026-fac0738f3b0a` with eleven completed tasks. The completion summary also records the automatically observed scheduled run and the 3,600-second schedule. This separation prevents the latest batch from being mistaken for the exact earlier verification event.

Table B.1: Primary local evidence used in the report

Path under project root	What it establishes
`results/last_run.json`	Latest batch state, counts and experiment results
`results/verification.json`	Database reconciliation and actual completed Prefect tasks
`results/completion_summary.json`	Saved thirty-test outcome and all-fifteen completion map
`results/eda/summary.json`	Before/after profile, optional repairs and outlier policy
`results/repaired_measurements.csv`	Audited optional measurements cleared to unknown
`results/quality/validation_report.html`	Per-expectation clean-data results
`results/quality/intentional_failure_report.html`	Expected failures on the separate damaged sample
`results/postgres/`	Window outputs and index/partition query plans

Path under project root	What it establishes
`results/olap/`	Three aggregation results and cross-engine comparisons
`results/mongo/crud_and_index_evidence.json`	Actual document operations and indexed query plan
`results/metrics/`	Generated-ticket views, football metrics and equality evidence
`results/delta_lab/summary.json`	Versioned operations, historical read and rejected writes
`results/benchmark/timings.csv`	Six measured storage/thread modes
`results/streaming/demo.json`	Run-specific valid, duplicate and rejected event counts
`results/prefect/`	Observed scheduled run and final deployment interval
`results/ui_checks.json`	Current five-tab website interaction checks

B.2 Figure Provenance

The architecture, quality-flow, relational and event-path diagrams were prepared from the active code. The website figures are screenshots of the running local football application. The exploration and benchmark figures are actual generated outputs from the saved project run. No screenshot from the unrelated e-commerce report is reused as football evidence.

B.3 Administrative Attachments

The certificate page contains a faculty-guide signature space for the institution to complete. No signatures, approval dates, similarity scores, publications or course certificates have been fabricated. Institutional verification documents may be appended after they are issued.